

GET LUCKY GOLF

Insurance-backed hole-in-one gaming

Developer Onboarding & Launch-at-Scale Brief

Scope of work for a contract / freelance engineer to help take the product from a working beta build to a stable, compliant public launch at scale.

Prepared by: Johannes Le Roux, Founder

Contact: johannes@getluckygolfclub.com

Date: 1 July 2026

Status: Working beta — PayFast in sandbox, not yet live to the public

Contents

1. Project snapshot	
2. What is already built and working	
3. Launch-at-scale gap analysis	
4. Scope of work	
5. Out of scope (for now)	
6. Deliverables & acceptance criteria	
7. Suggested phasing	
8. Ideal candidate	
9. Access, logistics & next steps	

1. Project snapshot

Get Lucky Golf is a mobile-first web app that lets golfers place an insurance-backed bet on any par-3: stake R50–R1,000 on a hole-in-one and win up to R1,000,000 if they sink an ace. **Indwe Risk Services (FSP 3425)** is the headline sponsor and insurer. The product is South-Africa-first: ZAR currency, en-ZA, PayFast payments.

Where we are today: a genuinely working beta. The full player journey, an admin back-office, native membership, and the payments/verification plumbing are all built and deployed on Vercel. A prior closed beta round has already happened. What remains is the hardening, compliance, observability, and operational maturity needed to run **real money at public scale** — that is the job this brief scopes.

Tech stack

Layer	Technology
Framework	Next.js 16 (App Router), React 19, TypeScript 5
Styling	Tailwind CSS v4, brand tokens (forest/gold/cream), Poster Gothic + Inter
Data / auth	Supabase (Postgres, Row-Level Security, Auth), Google sign-in
Payments	PayFast (once-off bets + recurring membership) — currently sandbox
Email	Resend (welcome email, internal lead notifications)
Hosting	Vercel (serverless / edge)
Testing	Vitest + Testing Library (one security-focused suite today)

Codebase scale: ~100 TypeScript/TSX files, 13 shared components, 26 API route handlers, 4 SQL migrations. Single Next.js repo, deployed on Vercel. This is a focused, readable codebase — not a sprawling legacy system. A strong engineer can be productive within days.

2. What is already built and working

The following are implemented and deployed. Treat these as the baseline to protect and harden — not to rebuild.

Player app

- Full onboarding funnel: splash → 18+ age gate → Google auth → profile onboarding → payment setup.
- Core play loop: select course → choose stake tier → confirm → PayFast checkout → record attempt → result (claim ace / miss) → hole-in-one claim + video verification.
- Six stake tiers, R50→R25k up to R1,000→R1,000,000, with server-side tier validation.
- Leaderboard, play history, and account management.
- Native in-app membership via PayFast recurring billing, with status surfaced in-app.
- Share-with-video on result pages (with iOS fallback handling).

Admin back-office

- Bets management, user management (incl. suspend), course & hole CRUD.
- Hole-in-one verification queue (single + batch actions).
- Revenue and payout reports, platform stats, and CSV export (with formula-injection escaping).

Payments & money integrity

- **PayFast ITN (webhook) handler** with three layers of authentication: MD5 signature verification, server-to-server "phone-home" validation, and source-IP allow-listing against PayFast's published ranges.
- Careful ITN design that **never clobbers a resolved play result** with a late payment confirmation.
- Idempotency guards on bet creation (unique payment reference + duplicate-insert handling).

Security & platform hygiene (already in place)

- Row-Level Security on Supabase tables; users can only touch their own profile.
- In-memory rate limiting on sensitive API routes (bets, payments, uploads, admin).
- A security test suite covering input validation, admin field-whitelisting (mass-assignment), CSV injection, batch limits, and PayFast signature generation.
- 18+ age-verification gate enforced server-side before any real-money bet.
- Brand system aligned to the live site (forest/gold/cream, Poster Gothic) and documented in a brand guide.

3. Launch-at-scale gap analysis

These are the gaps between "works in beta" and "safe to run real money for the public at scale." They are the heart of this engagement. Each is rated by priority and risk.

Area	Gap	Priority
Payment integrity	Bets are created from a client-supplied payment reference; the server does not independently confirm the payment actually completed before a bet goes "active." Needs a verified payment → bet state machine and a reconciliation ledger.	P0
Go-live config	PayFast is still in sandbox; live merchant credentials, return/notify URLs, and a staging vs production split need to be provisioned and tested end-to-end with real money.	P0
Rate limiting	Rate limiter is in-memory per serverless instance — it does not hold across Vercel's many concurrent instances, so it silently under-protects at scale. Needs a shared store (e.g. Upstash Redis).	P0
Observability	No error tracking, alerting, or structured logging — only console.log. At scale a failed payout or ITN failure could go unseen. Needs Sentry + alerting + a payments dashboard.	P0
Fraud / payout risk	Hole-in-one video verification is the single biggest fraud vector (it triggers real payouts). Needs a hardened claims workflow, audit trail, and anti-collusion controls.	P0
Compliance / legal	Real-money gaming in SA: confirm gambling/insurance regulatory posture with Indwe, FICA/RICA obligations, responsible-gambling controls, self-exclusion, and T&Cs/privacy at launch standard.	P0
CI/CD & migrations	No CI pipeline; SQL migrations are applied by hand in the Supabase console. Needs automated test/lint/build gates and managed, versioned migrations.	P1
Test coverage	One security unit suite exists; no end-to-end coverage of the money path and no load testing. Needs E2E on the critical flows and a basic load test.	P1
Exposure controls	No aggregate liability caps in code (per hole / per day / platform-wide). Confirm whether Indwe's policy covers this or whether the app must enforce limits.	P1
Scale & storage	Confirm Supabase connection pooling, video storage/CDN strategy, and image handling hold up under launch-day traffic spikes.	P1

Docs & onboarding	README is still the Next.js boilerplate. Needs a real runbook: env setup, deploy, rollback, on-call, incident response.	P2
Accessibility & polish	Final mobile/PWA polish, accessibility pass, and cross-device QA before public launch.	P2

Legend: P0 = must be done before taking public real money. P1 = required for a stable at-scale launch. P2 = launch-week polish and operational maturity.

4. Scope of work

The engagement is organised into four workstreams, sequenced roughly in the order below. Items map to the gap analysis above.

Workstream A — Money integrity & go-live (P0)

1. Design and implement a verified bet state machine: no bet becomes "active" until payment is independently confirmed server-side (via ITN and/or a PayFast query), not just because the client supplied a reference.
2. Build a payments/payout reconciliation ledger and an admin view to reconcile PayFast against internal bet and payout records.
3. Provision production PayFast (live merchant credentials, passphrase, notify/return/cancel URLs) and complete a real-money end-to-end test in a controlled way.
4. Stand up a proper staging vs production environment split with separate Supabase projects and secrets.

Workstream B — Reliability & observability (P0/P1)

1. Replace the in-memory rate limiter with a shared store (Upstash Redis or equivalent) so limits hold across serverless instances.
2. Add error tracking (Sentry), structured logging, and alerting on payment/ITN failures and payout events.
3. Build a lightweight operational dashboard / alerts for money-critical events (failed ITNs, mismatched amounts, stuck bets).
4. Introduce CI/CD: automated lint, type-check, test, and build on every change; managed, versioned database migrations.
5. Add end-to-end tests on the critical money path (stake → pay → bet → result → claim/payout) and a basic load test for launch-day traffic.

Workstream C — Fraud, risk & compliance (P0)

1. Harden the hole-in-one claims and video verification workflow: immutable audit trail, tamper-evidence, and anti-collusion controls on the payout trigger.

2. Implement configurable exposure/liability caps (per hole, per day, platform-wide) or confirm in writing that Indwe's policy removes the need.
3. Confirm and implement the regulatory/compliance posture with the founder and Indwe: gambling/insurance rules, FICA/RICA, responsible-gambling controls (deposit/loss limits, self-exclusion, cooling-off), and launch-grade T&Cs and privacy policy.

Workstream D — Scale, polish & handover (P1/P2)

1. Validate Supabase connection pooling, video/image storage and CDN strategy under load; fix any bottlenecks.
2. Final mobile/PWA polish, accessibility pass, and cross-device QA.
3. Write real project documentation: environment setup, deployment, rollback, incident response, and an on-call runbook.

5. Out of scope (for now)

- Native iOS/Android app store builds (the product is a mobile web app for launch).
- Rebuilding features that already work — the brief is to harden and scale, not to re-architect for its own sake.
- New product features beyond launch (e.g. tournaments, social features) unless separately agreed.
- Brand/visual redesign — the brand system is settled; the live site getluckygolf.co.za is the source of truth.

6. Deliverables & acceptance criteria

Deliverable	Accepted when...
Verified payment flow	A bet cannot reach "active" without server-confirmed payment; demonstrated with a failed-payment test that produces no active bet.
Reconciliation ledger	Every real-money transaction is reconciled between PayFast and internal records; discrepancies are visible in admin.
Production PayFast live	A real R50 bet completes end-to-end in production, is recorded correctly, and is reconciled.
Distributed rate limiting	Load test shows limits enforced consistently across concurrent serverless instances.
Observability	Sentry capturing errors; alerts fire on simulated ITN/payment failure; money-critical dashboard live.
CI/CD + migrations	Every change runs lint/type-check/test/build automatically; migrations are versioned and applied without manual SQL.

E2E + load tests	Automated tests cover the full money path and pass in CI; load test meets an agreed target (e.g. launch-day concurrency).
Fraud-hardened claims	Claims workflow has an immutable audit trail and documented anti-fraud controls, signed off by the founder.
Compliance controls	Responsible-gambling controls (limits, self-exclusion), FICA/RICA posture, and launch T&Cs/privacy are in place and reviewed.
Documentation & runbook	A new engineer can set up, deploy, roll back, and respond to an incident using the docs alone.

7. Suggested phasing

Indicative sequencing for a single contract engineer. Timeline is a planning guide, to be firmed up with the successful candidate.

Phase	Focus	Outcome
Phase 0	Onboarding & audit (week 1)	Get running locally, read the code, confirm/expand this gap list, agree the plan.
Phase 1	Money integrity & go-live (P0)	Verified payment flow, reconciliation, live PayFast in a controlled soft-launch.
Phase 2	Reliability & observability (P0/P1)	Distributed rate limiting, Sentry + alerting, CI/CD, E2E on the money path.
Phase 3	Fraud, risk & compliance (P0)	Hardened claims, exposure controls, responsible-gambling + compliance sign-off.
Phase 4	Scale, polish & handover (P1/P2)	Load validation, accessibility, docs/runbook — public launch readiness.

8. Ideal candidate

Must have

- Strong Next.js (App Router) + React + TypeScript, shipped to production.
- Real experience integrating a payment provider and reasoning about money integrity, webhooks, idempotency, and reconciliation.
- Postgres / Supabase (or similar) including Row-Level Security and migrations.
- Production instincts: observability, CI/CD, rate limiting, and incident response.

Nice to have

- Exposure to real-money / fintech / gaming or other regulated, money-handling products.

- PayFast or other South African payment rails; familiarity with SA compliance (FICA/RICA).
- Comfort owning a small codebase end-to-end and communicating clearly with a non-technical founder.

Ways of working

- This is real money and a regulated space — a careful, security-first, test-backed mindset matters more than raw speed.
- Small atomic changes, clear commits, and proof that money-critical changes work before they ship.

9. Access, logistics & next steps

What you'll be given on start

- Repository access (single Next.js repo) and a walkthrough of the codebase.
- Supabase project access (staging first), Vercel project access, and the required environment variables.
- PayFast sandbox credentials to start; production credentials provisioned when Workstream A is ready.
- Direct line to the founder (Johannes) and, where relevant, to Indwe for compliance questions.

First-week expectations

1. Get the app running locally against staging.
2. Read the code and confirm or challenge this gap analysis.
3. Come back with a prioritised, costed plan and any risks we've missed.

To express interest, contact johannes@getluckygolfclub.com with relevant experience — ideally a payments or regulated-money project you've shipped.

This document is confidential and prepared for the purpose of scoping the described engagement.